



Universidad Don Bosco, El Salvador

Asignatura: Desarrollo de Software para Móviles – DSM941

Docente: Ing. Alexander Alberto Siguenza Campos

Alumno: Téc. Isidro Alexander Marroquín Echeverría. (ME221443)

Actividad: Foro de Discusión 2.

Repositorio: <https://github.com/marroquin9953/DSM941-Foro2>

Fecha de entrega: 10 de mayo del 2026.

INTRODUCCIÓN.

¿Cuánto gasté este mes? Una pregunta simple que necesitaba una solución técnica real.

Cada día, millones de personas abren su billetera y no recuerdan en qué gastaron su dinero la semana pasada. No es un problema de voluntad: es un problema de herramientas. Este proyecto nació precisamente de esa fricción cotidiana, y su respuesta es una aplicación Android nativa construida para registrar, organizar y visualizar gastos personales en tiempo real.

El ecosistema elegido no fue casualidad. Kotlin, el lenguaje oficial de Google para Android desde 2019, ofrece una sintaxis que reduce el ruido del código sin sacrificar potencia, y elimina de raíz errores comunes como las excepciones por referencia nula. Sobre esa base, la aplicación se apoya en dos pilares de Firebase: autenticación de usuarios mediante correo o cuenta de Google a través de Firebase Authentication, y persistencia en la nube con sincronización en tiempo real gracias a Cloud Firestore.

El resultado es una app funcional donde el usuario puede iniciar sesión, registrar cada gasto con su nombre, monto, categoría y fecha, consultar su historial de transacciones y ver el total acumulado por mes, todo desde Android Studio, todo conectado a infraestructura de nivel empresarial.

INVESTIGACIÓN SOBRE AUTENTICACIÓN EN FIREBASE.

¿Qué es Firebase Authentication?

Firebase Authentication es un servicio de identidad administrado por Google que resuelve uno de los desafíos más frecuentes en el desarrollo de software moderno: gestionar el acceso de usuarios de manera segura sin necesidad de construir toda la infraestructura desde cero. A través de un SDK, el desarrollador delega en Google la responsabilidad de manejar el cifrado de contraseñas, la generación y rotación de tokens de sesión, los mecanismos de recuperación de cuentas y las defensas ante intentos de acceso malintencionado.

El servicio opera internamente con el estándar JSON Web Token (JWT), lo que garantiza compatibilidad con otros productos del ecosistema Firebase y con APIs de terceros. Cada usuario que completa el proceso de autenticación recibe un identificador único conocido como UID, el cual funciona como llave de acceso para vincular y proteger sus datos en servicios como Cloud Firestore o Realtime Database.

Tipos de autenticación disponibles

Firebase Authentication ofrece múltiples métodos de acceso que se adaptan al perfil de los usuarios y al propósito específico de cada aplicación. A continuación se describen los principales:

Correo electrónico y contraseña

Es el mecanismo de acceso más extendido y reconocido por los usuarios. El proceso consiste en que el usuario proporciona sus credenciales, Firebase las verifica y emite un token JWT. Las contraseñas se almacenan usando el algoritmo bcrypt, lo que impide que sean recuperadas en texto plano incluso por los administradores del proyecto. Su principal fortaleza es la universalidad: cualquier persona con una dirección de correo puede registrarse sin depender de plataformas externas. Como contrapartida, requiere que el usuario mantenga una contraseña segura y que la aplicación implemente un flujo de recuperación. Es especialmente

adecuado para plataformas educativas, sistemas internos y aplicaciones que necesitan control detallado del acceso.

Autenticación con Google (OAuth 2.0)

Este método utiliza el protocolo OAuth 2.0 para trasladar la verificación de identidad a los servidores de Google. El usuario es dirigido a una pantalla de consentimiento donde autoriza el acceso a su información básica de perfil; una vez concedido, Firebase recibe el token de Google y crea o asocia la cuenta en el proyecto. La principal ventaja es la eliminación de la fricción en el registro: el usuario no necesita crear ni recordar credenciales adicionales, y se beneficia automáticamente de las medidas de seguridad de Google, incluyendo la verificación en dos pasos. Su limitación es la dependencia de una cuenta de Google activa y conectividad a Internet. Resulta ideal para aplicaciones de consumo masivo y herramientas de productividad.

Autenticación por número de teléfono

Firebase envía un código de verificación de un solo uso (OTP) al número telefónico del usuario mediante SMS. Una vez ingresado correctamente, la sesión queda habilitada. Este método destaca por su accesibilidad, ya que prácticamente todos los usuarios móviles cuentan con un número de teléfono independientemente de si poseen cuentas en servicios en línea. Es particularmente útil en aplicaciones financieras, de salud o de entrega a domicilio donde verificar la identidad real es prioritario. Sus desventajas incluyen los costos variables del envío de SMS según el país, la dependencia de cobertura de red y la vulnerabilidad conocida como SIM swapping.

Proveedores federados (Facebook, Apple, GitHub, Microsoft, X)

Firebase admite la integración con diversos proveedores de identidad mediante OAuth 2.0 o SAML. Cada proveedor requiere configurar una aplicación en su portal correspondiente y registrar las credenciales en Firebase Console. La ventaja principal es la reducción del abandono durante el registro, ya que los usuarios pueden acceder con cuentas que ya utilizan cotidianamente. Cabe destacar que Sign in with Apple es obligatorio en aplicaciones iOS que ofrezcan otras opciones

de inicio de sesión federado, según las políticas de la App Store. La complejidad de configuración y las restricciones propias de cada proveedor son las limitaciones más relevantes.

Autenticación anónima

Firebase permite generar sesiones temporales para usuarios que aún no desean registrarse formalmente. Se crea una cuenta con un UID único que otorga acceso a los recursos configurados en las reglas de seguridad. En cualquier momento, esta cuenta anónima puede vincularse a un método de autenticación permanente sin perder los datos ya generados. Es una solución ideal para aplicaciones que buscan ofrecer una experiencia previa al registro, como juegos móviles o tiendas con carrito de compras antes del checkout. La limitación más importante es que la cuenta se elimina si el usuario desinstala la aplicación sin haber completado el proceso de vinculación.

Enlace por correo electrónico (Passwordless)

En lugar de solicitar una contraseña, Firebase envía al correo del usuario un enlace de autenticación de un solo uso con validez limitada. Al acceder desde el mismo dispositivo, la sesión se abre de forma automática. Este enfoque elimina los riesgos asociados a contraseñas débiles o reutilizadas, lo que lo convierte en una opción robusta para aplicaciones donde la seguridad es crítica. Su limitación principal es que el usuario debe tener acceso a su correo en el momento exacto en que desea autenticarse, lo que puede representar una barrera en contextos con conectividad intermitente.

Ventajas generales de Firebase Authentication

Adoptar Firebase Authentication supone beneficios concretos para equipos de cualquier tamaño. En primer lugar, comprime drásticamente los tiempos de desarrollo: lo que en un sistema propio tomaría semanas de trabajo se implementa en pocas horas. Además, el servicio gestiona de forma automática aspectos críticos como el cifrado de contraseñas, la renovación de tokens y la protección contra ataques de enumeración de cuentas.

Su integración nativa con Cloud Firestore y Realtime Database permite definir reglas de seguridad basadas en el UID del usuario, garantizando que cada persona solo tenga acceso a su propia información. Por último, la escalabilidad del servicio es uno de sus activos más sólidos: puede operar desde un prototipo con decenas de usuarios hasta una plataforma con millones, sin que el equipo de desarrollo deba gestionar la infraestructura subyacente.

Limitaciones y consideraciones

A pesar de sus fortalezas, Firebase Authentication presenta restricciones que deben evaluarse antes de adoptarlo. La más significativa es la dependencia de los servidores de Google: si el servicio presenta una interrupción, los usuarios no podrán iniciar sesión. Aunque Google publica un SLA del 99.95% de disponibilidad, esta dependencia puede ser inaceptable en entornos con regulaciones estrictas de soberanía de datos.

Otro aspecto a considerar es la personalización de la interfaz de usuario. Las pantallas prediseñadas de FirebaseUI facilitan la implementación inicial, pero pueden no alinearse con los estándares de marca de ciertos proyectos, lo que obliga al equipo a construir su propia interfaz utilizando directamente el SDK. Finalmente, el método de verificación por teléfono implica costos que escalan con el volumen de usuarios y varían considerablemente según el país de destino.

Flujo técnico de autenticación

Independientemente del método seleccionado, el proceso de autenticación sigue una secuencia lógica y segura. En primer lugar, el cliente transmite las credenciales al SDK de Firebase. El SDK las comunica al servidor de Firebase Authentication mediante una conexión HTTPS cifrada. Firebase verifica las credenciales con el proveedor de identidad correspondiente y, si la validación es exitosa, emite un token JWT firmado con una clave privada de Google. El cliente almacena este token de forma segura y lo incluye en cada solicitud posterior a los servicios de Firebase.

Finalmente, servicios como Firestore validan el token antes de conceder acceso a los datos del usuario.

Este flujo garantiza que las credenciales nunca sean procesadas directamente por el código de la aplicación, sino que pasen exclusivamente por la infraestructura segura de Google. El token tiene una vigencia de una hora y Firebase lo renueva de forma automática cuando está próximo a vencer, asegurando una experiencia fluida sin necesidad de que el usuario vuelva a iniciar sesión frecuentemente.

FLUJO DE APLICACION.



